Dhirubhai Ambani University
Technology

Formerly DA-IICT

Hybrid Post-Quantum Cryptographic File Transfer System

Course: Introduction to Cryptography

Using ECDH, Kyber (ML-KEM), and AES

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Background . . . 3
- 1.3 Project Objectives . . . 3

2 Elliptic Curve Diffie–Hellman (ECDH) **3**
- 2.1 Core Idea . . . 4
- 2.2 Elliptic Curve Mathematics . . . 4
  - 2.2.1 Curve Equation . . . 4
  - 2.2.2 Points on the Curve . . . 4
- 2.3 Basic Point Operations . . . 4
- 2.4 ECDH Protocol Steps . . . 4
- 2.5 Why It Is Secure . . . 5
- 2.6 Small Numerical Example . . . 5

3 Lattice Theory Fundamentals **5**
- 3.1 What is a Lattice? . . . 5
  - 3.1.1 Simple 2D Example . . . 5
- 3.2 The Shortest Vector Problem (SVP) . . . 5
  - 3.2.1 Example . . . 6
  - 3.2.2 Why SVP Becomes Hard . . . 6

4 Learning With Errors (LWE) **6**
- 4.1 Mathematical Form . . . 6
- 4.2 Why Kyber Uses Polynomials . . . 6

5 Module-LWE **7**
- 5.1 The Ring R_q . . . 7
  - 5.1.1 The Ring $\mathbb{Z}[X]$ . . . 7
  - 5.1.2 Modulo Polynomial $(X^n + 1)$ . . . 7
  - 5.1.3 Modulo Integer q . . . 7
- 5.2 Numerical Example . . . 7
- 5.3 What the Attacker Sees . . . 8

6 Compression and Decompression in Kyber **8**
- 6.1 Core Idea . . . 8
- 6.2 The Compression Function . . . 8
  - 6.2.1 Example . . . 9
- 6.3 The Decompression Function . . . 9
- 6.4 Why This Works in Cryptography . . . 9

7 Binomial Distribution B_η in Kyber **9**
- 7.1 Definition of B_η . . . 9
- 7.2 Possible Values of $(a_i - b_i)$ . . . 10
- 7.3 Key Observations . . . 10

8 IND-CPA Security **10**
8.1 Algorithm 1 — Key Generation . . . 10
8.2 Algorithm 2 — Encryption . . . 11
8.2.1 Message Encoding . . . 11
8.3 Algorithm 3 — Decryption . . . 11
8.4 Why Decryption Works Mathematically . . . 11
8.5 Why the Message is Recovered Correctly . . . 12
8.6 Correctness Condition . . . 12
8.6.1 Why $q/4$? . . . 12

9 Key Encapsulation Mechanism (KEM) **12**
9.1 Why Use a KEM? . . . 13
9.2 The CCA-Secure KEM . . . 13
9.2.1 Algorithm 4 — Encapsulation . . . 13
9.2.2 Algorithm 5 — Decapsulation . . . 13

10 Parameters and Security **14**
10.1 What "161 Bits of Security" Means . . . 14
10.2 Why $n = 256$? . . . 14
10.3 Why $k = 3$? . . . 14

11 Hybrid Key Derivation **14**
11.1 Overview . . . 14
11.2 Notation . . . 15
11.3 Key Derivation Formula . . . 15
11.4 Algorithm — Hybrid Key Derivation . . . 15
11.5 Why SHA-256? . . . 15
11.6 Security Guarantee . . . 15
11.7 Comparison with Single-Scheme Approaches . . . 16

12 Project Website and Demonstration **16**
12.1 Live Web Application . . . 16

1 Introduction

1.1 Motivation

The rise of quantum computing poses a direct threat to the cryptographic systems that secure modern digital communication. Algorithms such as RSA and Elliptic Curve Diffie–Hellman (ECDH), which underpin the majority of today's encrypted traffic, can be broken in polynomial time by Shor's algorithm running on a sufficiently powerful quantum computer.

This project addresses that threat by designing and implementing a **hybrid file-transfer system** that combines classical and post-quantum cryptography, ensuring security even if one of the two components is compromised.

1.2 Background

Lattice-based cryptography offers strong post-quantum security guarantees rooted in the hardness of geometric problems such as the *Shortest Vector Problem* (SVP). The **Learning With Errors** (LWE) problem, introduced by Regev in 2005, is the central building block:

1. Generate a public matrix A and a secret vector $\mathbf{s}$.
2. Compute $\mathbf{b} = A\mathbf{s} + \mathbf{e}$, where $\mathbf{e}$ is a small noise vector.
3. Given only A and $\mathbf{b}$, recovering $\mathbf{s}$ is computationally infeasible — even for a quantum adversary.

Kyber (standardised as **ML-KEM** by NIST in 2024) builds on a structured variant of LWE called Module-LWE, and is the post-quantum algorithm implemented in this project.

1.3 Project Objectives

This project delivers the following:

#	Objective
1	Explain the mathematical foundations of lattice-based cryptography
2	Derive and analyse the LWE problem and its security properties
3	Study the Module-LWE variant used in Kyber (ML-KEM)
4	Review classical ECDH key exchange
5	Implement a hybrid file-transfer tool combining Kyber and ECDH
6	Provide a formal security analysis and reduction for each scheme

The hybrid design ensures **forward security**: even if the classical ECDH layer is broken by a quantum computer, the Kyber layer remains secure, and vice versa.

2 Elliptic Curve Diffie–Hellman (ECDH)

ECDH (Elliptic Curve Diffie–Hellman) is a key agreement protocol that allows two parties to generate the same shared secret over an insecure channel. It is based on the difficulty of the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**.

2.1 Core Idea

Alice and Bob want to share a secret without directly sending it. They exchange only public keys, then both compute the same value:

$$\text{Shared secret} = abG,$$

where a is Alice's private key, b is Bob's private key, and G is a public base point on an elliptic curve. Because

$$a(bG) = b(aG) = abG.$$

2.2 Elliptic Curve Mathematics

All calculations are done over a finite field $\mathbb{F}_p = \{0, 1, \ldots, p-1\}$ with a prime p. Arithmetic is performed modulo p.

2.2.1 Curve Equation

$$y^2 \equiv x^3 + ax + b \pmod{p},$$

with $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$ (to avoid singularities).

2.2.2 Points on the Curve

A point is a pair (x, y) satisfying the equation, together with a special point O called the *point at infinity*.

2.3 Basic Point Operations

- **Point addition** $P + Q$: for two distinct points $P = (x_1, y_1)$, $Q = (x_2, y_2)$,

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

- **Point doubling** $2P$: for $P = (x_1, y_1)$,

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

- **Scalar multiplication** kP: adding P to itself k times. This is computed efficiently (e.g., double-and-add), but the exact algorithm is not needed for understanding the protocol.

2.4 ECDH Protocol Steps

1. **Public parameters**: A curve $E(\mathbb{F}_p)$, a base point G, and its order n are publicly known.
2. **Alice** picks a random private key $a \in [1, n-1]$ and computes her public key $A = aG$.
3. **Bob** picks a random private key $b \in [1, n-1]$ and computes his public key $B = bG$.
4. They exchange A and B.
5. Alice computes $S = aB = a(bG) = abG$.
6. Bob computes $S = bA = b(aG) = abG$.
7. Both now share the same point S. The shared secret is derived from its x-coordinate (e.g., using a simple hash).

2.5 Why It Is Secure

An attacker sees G, $A = aG$, and $B = bG$. To obtain abG, they would need to find a from A (or b from B). This is the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**, which is believed to be hard for classical and quantum computers when the curve is chosen appropriately.

2.6 Small Numerical Example

Let the curve be $y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$, and choose $G = (5, 1)$.

- Alice takes $a = 3$: $A = 3G = (10, 6)$.
- Bob takes $b = 7$: $B = 7G = (0, 6)$.
- Shared secret: Alice computes $3B = 21G$, Bob computes $7A = 21G$. Both obtain the same point.

The x-coordinate of that point is then used as the basis for a symmetric key.

3 Lattice Theory Fundamentals

3.1 What is a Lattice?

A lattice is simply a grid of points in space created using vectors. Mathematically, given vectors

$$\mathbf{v}_1,\ \mathbf{v}_2,\ \ldots,\ \mathbf{v}_n,$$

any lattice point is

$$a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + \cdots + a_n\mathbf{v}_n,$$

where $a_1, a_2, \ldots$ are integers. So a lattice is just all integer combinations of some vectors.

3.1.1 Simple 2D Example

Let $\mathbf{v}_1 = (2, 0)$ and $\mathbf{v}_2 = (0, 2)$. Any lattice point has the form $a_1(2, 0) + a_2(0, 2)$:

a_1	a_2	**Point**
0	0	$(0, 0)$
1	0	$(2, 0)$
0	1	$(0, 2)$
1	1	$(2, 2)$
2	1	$(4, 2)$

Imagine graph paper. If you start at the origin and keep adding certain vectors again and again, you create many points. All those points together form a lattice.

3.2 The Shortest Vector Problem (SVP)

Definition 3.1 (SVP)**.** The Shortest Vector Problem (SVP) is to find the shortest non-zero vector in the lattice.

Now suppose we have many lattice points. The task is to find the lattice point closest to the origin (excluding the origin itself).

3.2.1 Example

Let $\mathbf{v}_1 = (4, 1)$ and $\mathbf{v}_2 = (1, 3)$. Some candidate lattice points and their lengths:

a_1	a_2	**Point**	**Length**
1	0	$(4, 1)$	$\sqrt{16+1} = \sqrt{17} \approx 4.12$
0	1	$(1, 3)$	$\sqrt{1+9} = \sqrt{10} \approx 3.16$
1	1	$(5, 4)$	$\sqrt{25+16} = \sqrt{41} \approx 6.40$
1	-1	$(3, -2)$	$\sqrt{9+4} = \sqrt{13} \approx 3.61$
2	0	$(8, 2)$	$\sqrt{64+4} = \sqrt{68} \approx 8.25$

Shortest vector = $(1, 3)$ with length $\sqrt{10}$. This is the SVP solution.

3.2.2 Why SVP Becomes Hard

In 2D we can try all combinations. But in 500 dimensions there are trillions of possibilities. Even supercomputers cannot easily solve it. This is why lattice cryptography is believed to be **quantum resistant**.

4 Learning With Errors (LWE)

LWE uses lattices to build encryption by hiding a secret vector inside linear equations with noise.

4.1 Mathematical Form

$$b = As + e$$

Symbol	**Meaning**
A	Public matrix
s	Secret vector
e	Small noise
b	Public vector

The attacker sees (A, b) but must find s. Without noise it is easy; with noise it becomes extremely hard.

4.2 Why Kyber Uses Polynomials

Normal LWE uses matrices and vectors. For example:

$$b = As + e.$$

But large matrices are slow and inefficient for practical cryptography. Therefore, Kyber replaces numbers with polynomials. Instead of working with vectors like $(3, 5, 2)$, we use polynomials such as

$$3 + 5x + 2x^2.$$

Operations on polynomials can be computed very quickly using FFT-like methods (Number Theoretic Transforms). This makes Kyber both secure and fast enough for real-world applications.

5 Module-LWE

Kyber uses a polynomial version of $b = As + e$, where:

Symbol	Meaning
A	Polynomial matrix
s	Secret polynomial vector
e	Small noise polynomial

5.1 The Ring R_q

A ring is just a set of numbers where you can add, subtract, and multiply, and everything still stays inside the set. In Kyber, computations are performed in the ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1).$$

5.1.1 The Ring $\mathbb{Z}[X]$

$$\mathbb{Z}[X] = \{ a_0 + a_1X + a_2X^2 + \cdots + a_kX^k \mid a_i \in \mathbb{Z} \}$$

Symbol	Meaning
$\mathbb{Z}$	Integers
X	Variable
Coefficients	Are integers

5.1.2 Modulo Polynomial $(X^n + 1)$

We define $\mathbb{Z}[X]/(X^n + 1)$. This means:

Rule	Effect
$X^n \equiv -1$	Powers of X wrap around with a sign flip
$X^{n+k} \equiv -X^k$	Any higher power reduces using the rule above
$\deg(f(X)) < n$	All polynomials stay below degree n

5.1.3 Modulo Integer q

$$\mathbb{Z}_q = \{0, 1, 2, \ldots, q-1\}, \qquad a \equiv b \pmod{q} \text{ if } q \mid (a - b).$$

Whenever a term exceeds degree $n - 1$, it is wrapped around and multiplied by -1. This ensures that all resulting polynomials have degree less than n, preventing growth during multiplication. Together with reducing coefficients modulo q, all operations remain within a fixed finite space, enabling efficient and secure computation.

5.2 Numerical Example

Assume arithmetic modulo 7 and polynomials of degree ≤ 1 (i.e., working in $\mathbb{Z}_7[x]/(x^2 + 1)$, so $x^2 \equiv -1$).

$$A = \begin{bmatrix} 1 + x & 2 \\ x & 3 \end{bmatrix}, \qquad s = \begin{bmatrix} 1 + x \\ 4 \end{bmatrix}, \qquad e = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Step 1 — Compute As:

$$As = \begin{bmatrix} (1+x)(1+x) + 2 \cdot 4 \\ x(1+x) + 3 \cdot 4 \end{bmatrix} = \begin{bmatrix} (1 + 2x + x^2) + 8 \\ (x + x^2) + 12 \end{bmatrix}.$$

Step 2 — Replace $x^2 \to -1$:

$$(1 + 2x - 1) + 8 = 2x + 8, \qquad (x - 1) + 12 = x + 11.$$

Step 3 — Reduce coefficients mod 7:

$$As \equiv \begin{bmatrix} 2x + 1 \\ x + 4 \end{bmatrix} \pmod 7.$$

Step 4 — Add noise e:

$$b = As + e = \begin{bmatrix} (2x + 1) + 1 \\ (x + 4) - 1 \end{bmatrix} = \begin{bmatrix} 2x + 2 \\ x + 3 \end{bmatrix} \pmod 7.$$

5.3 What the Attacker Sees

The attacker has access to A and b, but must find s. Because of noise e, solving becomes extremely difficult at realistic Kyber parameters:

Parameter	Value (real Kyber)
Dimension	≈ 256
Polynomial degree	≈ 256
Modulus	$q \approx 3329$

This hardness relates to lattice problems like the Shortest Vector Problem.

6 Compression and Decompression in Kyber

6.1 Core Idea

Operation	Effect
Compression	Shrink a number to fewer bits (lose some precision)
Decompression	Expand it back (approximate original)

6.2 The Compression Function

$$\text{Compress}_q(x, d) = \left\lceil \frac{2^d}{q} \cdot x \right\rceil \bmod {}^{+}2^d,$$

where $\bmod^+$ returns a value in $\{0, 1, \ldots, 2^d - 1\}$.

6.2.1 Example

Let $q = 100$, $d = 3$, and $x = 42$:

Step	Value
2^d	$2^3 = 8$
$\frac{2^d}{q}$	$\frac{8}{100} = 0.08$
0.08×42	$= 3.36$
$\lceil 3.36 \rceil$	$= 4$
$4 \bmod {}^{+}8$	$= 4$

Hence $\text{Compress}_{100}(42, 3) = 4$.

6.3 The Decompression Function

$$\text{Decompress}_q(x, d) = \left\lceil \frac{q}{2^d} \cdot x \right\rceil.$$

Continuing the example:

$$\frac{q}{2^d} = \frac{100}{8} = 12.5, \qquad 12.5 \times 4 = 50, \qquad \lceil 50 \rceil = 50.$$

So we turned 42 into 4, then back into 50. We lost some precision ($42 \to 50$), but saved space!

6.4 Why This Works in Cryptography

The error is small. Kyber is designed so that small errors do *not* break decryption, but they reduce data size by a lot.

7 Binomial Distribution B_η in Kyber

The binomial distribution is a way to generate small random numbers centered around 0.

7.1 Definition of B_η

$$B_\eta : \quad \text{Sample } \{(a_i, b_i)\}_{i=1}^{\eta} \text{ uniformly from } \{0,1\}^2, \quad \text{output } \sum_{i=1}^{\eta} (a_i - b_i).$$

Repeat the following η times:

1. Pick two random bits $a_i, b_i \in \{0, 1\}$.
2. Compute $a_i - b_i$.
3. Add the result to a running sum.

7.2 Possible Values of $(a_i - b_i)$

a_i	b_i	$a_i - b_i$
1	0	+1
0	1	−1
1	1	0
0	0	0

So each step contributes either $+1$, -1, or 0. After η repetitions, the final sum lies between $-\eta$ and $+\eta$.

7.3 Key Observations

- Most of the time, the sum stays close to 0.
- Large absolute values are rare when η is small.
- Higher probability for $0, \pm 1$; lower probability for $-\eta$ and $+\eta$.

This is exactly what we want in cryptography: small, balanced noise that does not disrupt decryption but provides security.

8 IND-CPA Security

IND-CPA (Indistinguishability under Chosen Plaintext Attack) means:

> Even if an attacker can choose two different messages and see the encryption of one of them, they cannot tell which message was encrypted.

8.1 Algorithm 1 — Key Generation

Step	Operation
1	$\rho, \sigma \leftarrow \{0,1\}^{256}$ *Pick two random seeds*
2	$A \leftarrow \mathrm{Sam}(\rho)$ *Generate random matrix from seed ρ*
3	$(s, e) \leftarrow \beta_k^\eta$ *Sample secret and error from distribution*
4	$t \leftarrow \mathrm{Compress}_q(A \cdot s + e,\ d_t)$ *Compute and compress public key*
5	**return** $\big(\mathrm{pk} = (t, \rho),\ \mathrm{sk} = s\big)$

8.2 Algorithm 2 — Encryption

Step	Operation
1	$r \leftarrow \{0,1\}^{256}$ *Random encryption seed*
2	$t \leftarrow \mathrm{Decompress}_q(t,\ d_t)$ *Uncompress public key*
3	$A \leftarrow \mathrm{Sam}(\rho)$ *Regenerate same matrix A*
4	$(r, e_1, e_2) \leftarrow \beta_k^\eta$ *Sample noise for encryption*
5	$u \leftarrow \mathrm{Compress}_q(A^\top \cdot r + e_1,\ d_u)$ *First ciphertext part*
6	$v \leftarrow \mathrm{Compress}_q\big(t^\top \cdot r + e_2 + \lceil q/2 \rceil \cdot m,\ d_v\big)$ *Second part with message*
7	**return** $c = (u, v)$

8.2.1 Message Encoding

The term $\lceil q/2 \rceil \cdot m$ encodes the message bit m:

$$\lceil q/2 \rceil \cdot m = \begin{cases} 0 & \text{if } m = 0, \\ \lceil q/2 \rceil & \text{if } m = 1. \end{cases}$$

($\lceil q/2 \rceil$ is about half of q.)

8.3 Algorithm 3 — Decryption

Step	Operation
1	$u \leftarrow \mathrm{Decompress}_q(u,\ d_u)$
2	$v \leftarrow \mathrm{Decompress}_q(v,\ d_v)$
3	**return** $\mathrm{Compress}_q\big(v - s^\top \cdot u,\ 1\big)$

8.4 Why Decryption Works Mathematically

From the encryption procedure (ignoring compression noise for clarity):

$$u \approx A^\top \cdot r + e_1, \qquad v \approx t^\top \cdot r + e_2 + \left\lceil \tfrac{q}{2} \right\rceil \cdot m.$$

From key generation: $t = A \cdot s + e$.

Step 1 — Substitute u and v:

$$v - s^\top u \approx \left(t^\top r + e_2 + \left\lceil \tfrac{q}{2} \right\rceil m\right) - s^\top \left(A^\top r + e_1\right).$$

Step 2 — Expand:

$$= t^\top r + e_2 + \left\lceil \tfrac{q}{2} \right\rceil m - s^\top A^\top r - s^\top e_1.$$

Step 3 — Replace $t = As + e$, so $t^\top = s^\top A^\top + e^\top$:

$$t^\top r = s^\top A^\top r + e^\top r.$$

Step 4 — Substitute:

$$v - s^\top u \approx \left(s^\top A^\top r + e^\top r\right) + e_2 + \left\lceil \tfrac{q}{2} \right\rceil m - s^\top A^\top r - s^\top e_1.$$

Step 5 — Cancel $s^\top A^\top r$:

$$\boxed{v - s^\top u \approx \underbrace{e^\top r + e_2 - s^\top e_1}_{\text{small noise}} + \left\lceil \frac{q}{2} \right\rceil m.}$$

8.5 Why the Message is Recovered Correctly

The terms $e^\top r$, e_2, and $s^\top e_1$ are all *small* (drawn from β_η). If the total noise is less than $q/4$:

Message	$v - s^\top u$ **is close to . . .**
$m = 0$	0
$m = 1$	$\lceil q/2 \rceil$

The final $\mathrm{Compress}_q(\cdot, 1)$ rounds values near 0 to 0 and values near $q/2$ to 1, recovering the original message bit m.

8.6 Correctness Condition

Decryption works correctly when:

$$\left\| e^\top r + e_2 + c_v + c_t^\top r - s^\top e_1 - s^\top c_u \right\|_\infty < \left\lceil \frac{q}{4} \right\rceil.$$

The left-hand side is the total noise from sampling errors (e, e_1, e_2) and compression artifacts (c_v, c_t, c_u). For decryption to succeed, this must be *strictly smaller than* $\lceil q/4 \rceil$.

8.6.1 Why $q/4$?

The decryption decision distinguishes between:

$$\begin{cases} 0 + \text{noise} & (\text{message } m = 0) \\ \dfrac{q}{2} + \text{noise} & (\text{message } m = 1) \end{cases}$$

If the noise magnitude is less than $q/4$:

- $0 + \text{noise}$ is closer to 0 than to $q/2$.
- $\dfrac{q}{2} + \text{noise}$ is closer to $q/2$ than to 0.

Hence rounding via $\mathrm{Compress}_q(\cdot, 1)$ correctly recovers the message bit.

9 Key Encapsulation Mechanism (KEM)

A **Key Encapsulation Mechanism (KEM)** is a method to securely send a random secret key from a sender to a receiver using public-key cryptography.

9.1 Why Use a KEM?

Public-key encryption is slow for large data. KEM solves this by:

1. Using public-key cryptography to securely send a small random key.
2. Using that key with a fast symmetric cipher (AES) to encrypt the actual data.

Real-world examples: HTTPS, secure messaging, VPNs.

9.2 The CCA-Secure KEM

CCA means **Chosen Ciphertext Attack** — even if attackers can trick you into decrypting some messages, they still cannot break the system.

9.2.1 Algorithm 4 — Encapsulation

Step	Operation
1	$m \leftarrow \{0,1\}^{256}$ *Random message*
2	$(\hat{H}, r) \leftarrow G(H(pk), m)$ *Derive randomness from message*
3	$(u, v) \leftarrow \text{Kyber.CPA.Enc}((t, p),\ m;\ r)$ *Encrypt using m*
4	$c \leftarrow (u, v)$ *Ciphertext*
5	$K \leftarrow H(\hat{H}, H(c))$ *Final shared key*
6	**return** (c, K)

9.2.2 Algorithm 5 — Decapsulation

Step	Operation
1	$m' \leftarrow \text{Kyber.CPA.Dec}(s,\ (u, v))$ *Try to decrypt*
2	$(\hat{H}', r') \leftarrow G(H(pk), m')$ *Recompute what encryption would be*
3	$(u', v') \leftarrow \text{Kyber.CPA.Enc}((t, p),\ m';\ r')$ *Re-encrypt*
4	**if** $(u', v') = (u, v)$ **then** $K \leftarrow H(\hat{H}', H(c))$
5	**else** $K \leftarrow H(z, H(c))$ *Return pseudo-random key instead*
6	**end if**

The clever part (implicit rejection): If the ciphertext is invalid, the decapsulation does *not* return "error" (which could leak information). Instead, it returns a pseudo-random key stored in the secret key. This prevents attackers from learning anything about the secret key. So the secret key is $sk = (s, z)$, where s is used for decryption and z is a backup secret for security.

10 Parameters and Security

Parameter	Value	What it means
n	256	Polynomial degree (how many coefficients)
k	3	Module dimension (matrix size: 3×3)
q	7681	Modulus (prime number)
η	4	Noise distribution parameter
(d_u, d_v, d_t)	$(11, 3, 11)$	Compression bits
δ	2^{-142}	Decryption failure probability
pq-sec	161	Estimated post-quantum security (bits)

10.1 What "161 Bits of Security" Means

An attacker would need about 2^{161} operations to break Kyber:

$$2^{161} \approx 2.9 \times 10^{48} \text{ operations.}$$

Even with the fastest quantum computers imaginable, this is impossible.

10.2 Why $n = 256$?

They want to encapsulate 256 bits of entropy (the shared key). Each polynomial coefficient can carry 1 bit of message, so they need 256 coefficients.

10.3 Why $k = 3$?

This is the "flexibility" advantage. The ring ($n = 256$) stays fixed, and k is changed to adjust security:

k	Security	Use case
2	102-bit	Faster, smaller
3	161-bit	Recommended
4	218-bit	"Paranoid" mode

Analogy: Imagine a puzzle. Ring-LWE ($k = 1$) is like a 1-layer puzzle. Module-LWE with $k = 3$ is like a 3-layer puzzle — much harder to solve, but you can still use the same puzzle pieces!

11 Hybrid Key Derivation

11.1 Overview

Both ECDH and Kyber independently produce a shared secret during the key-exchange phase. Rather than trusting a single scheme, the hybrid design *combines* both secrets into one symmetric key using a cryptographic hash function. This ensures that the final key is secure as long as *at least one* of the two underlying schemes remains unbroken.

11.2 Notation

Symbol	Meaning
K_{ECDH}	Shared secret produced by ECDH (derived from x-coordinate of abG)
K_{Kyber}	Shared secret produced by Kyber KEM encapsulation/decapsulation
$\|\|$	Byte concatenation
K	Final 256-bit hybrid symmetric key

11.3 Key Derivation Formula

The hybrid key K is computed as:

$$\boxed{K = \text{SHA-256}(K_{\text{ECDH}} \,\|\, K_{\text{Kyber}})}$$

where $K_{\text{ECDH}} \| K_{\text{Kyber}}$ denotes the byte-level concatenation of the two shared secrets, and SHA-256 is the NIST-standardised 256-bit hash function.

11.4 Algorithm — Hybrid Key Derivation

Step	Operation
1	Run ECDH protocol $\Rightarrow$ obtain K_{ECDH}
2	Run Kyber KEM $\Rightarrow$ obtain K_{Kyber}
3	Concatenate: $C \leftarrow K_{\text{ECDH}} \,\|\, K_{\text{Kyber}}$
4	Hash: $K \leftarrow \text{SHA-256}(C)$
5	Use K as the AES symmetric key for file encryption

11.5 Why SHA-256?

- **Fixed output length**: SHA-256 always produces exactly 256 bits, perfectly matching the AES-256 key size.
- **One-way**: Given K, it is computationally infeasible to recover K_{ECDH} or K_{Kyber}.
- **Collision resistance**: No two distinct inputs produce the same output hash.
- **Avalanche effect**: A single bit change in either shared secret completely changes K.

11.6 Security Guarantee

The hybrid key K is secure under the following condition:

*An adversary can break the hybrid scheme only if they break **both** ECDH **and** Kyber simultaneously.*

This gives us **forward security** in the quantum setting:

- A classical attacker cannot break Kyber (lattice hardness).
- A quantum attacker (running Shor's algorithm) can break ECDH, but *still cannot break Kyber*.
- Therefore, K remains secure against both classical and quantum adversaries.

11.7 Comparison with Single-Scheme Approaches

Scheme	Classical security	Quantum security	Used here
ECDH only	✓	×	No
Kyber only	✓	✓	No
Hybrid (ECDH + Kyber)	✓	✓	**Yes**

12 Project Website and Demonstration

12.1 Live Web Application

To demonstrate our implementation, we have developed a fully functional web-based interface for the hybrid cryptographic file transfer system.

Click here to access the project website